

Simulation and data analysis for an optics experiment about spatial coherence

Project for the exam of the course
"Scientific Python"

Leonardo ROSSI

Contents

1	Introduction	3
1.1	Set-up	3
1.2	Simulation	4
2	Review of the physics	5
2.1	Speckle fields	5
2.2	Measuring the field correlation function	5
2.3	Finite slit width effects	6
2.4	Computing the field correlation function	7
3	Simulation	8
3.1	Generation of the speckle fields	8
3.2	Spatial filtering	8
3.3	Interference patterns	9
3.4	Data analysis	10
3.5	Visibility curve and phase	11
3.6	Relation between Δl and Δk	11
A	Libraries	13
A.1	Brief overview of Dash	13
A.2	Notable comments on the implementation	14

1 Introduction

For this project, I wrote a simulation of an experiment and the code for analysis of the generated data, controlled by a GUI. The experiment concerns the (second order) statistics of a time independent speckle field, i.e. its spatial coherence. The statistical properties of the field can be controlled by a spatial filtering, and can be measured by interferometry, thus verifying the inverse proportionality relation between the width of the spatial spectrum and the correlation length predicted by the Wiener-Khintchine theorem.

1.1 Set-up

The arrangement of optical components in the main part of the experiment is as follows.

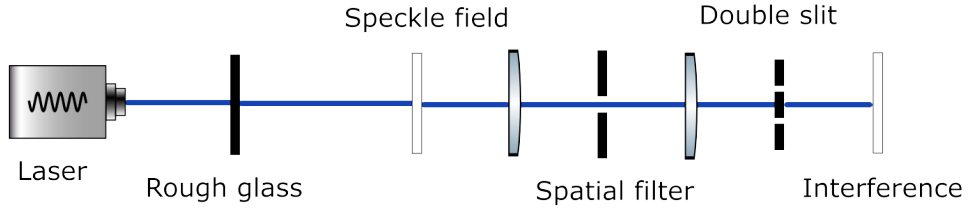


Figure 1: Set-up

The laser beam is assumed to have been previously expanded, filtered and collimated. The speckle field is produced by a rough glass surface, and is subsequently spatially filtered by a $4f$ set-up with a rectangular slit. Before filtering, the field already has a nonzero correlation length δ given by the Van Cittert-Zernike relation $\delta = z\lambda/D$ (where z is the longitudinal propagation distance, λ the wavelength and D the diameter of the source, i.e. the collimated beam), which however is assumed to be negligible in comparison with the correlation induced by the filtering.

After filtering, the field has a (field) correlation function which is given by the Fourier transform of the filtered spectrum (Wiener-Khintchine theorem), which is

$$\gamma(s) = \text{sinc} \left(\frac{\pi b s}{\lambda f} \right) \quad (1.1)$$

for a rectangular filtering slit of width b (here f is the focal length of the lenses used for the filter), or

$$\gamma(s) = \exp \left[-2 \left(\frac{\pi b s}{\lambda f} \right)^2 \right]. \quad (1.2)$$

for a gaussian filtering.

This correlation function can be measured by Young interferometry: if the filtered field is probed by a double slit with separation s , one has the relation

$$\mathcal{V} = |\gamma(s)|, \quad (1.3)$$

where $\mathcal{V}$ is the visibility, or contrast, of the interference pattern (the pattern is supposed to be averaged over an ensemble of speckle fields in order to capture the statistical properties). Once obtained the absolute value of $\gamma(s)$, the phase (which is always a ± 1) can be inferred by the form of the pattern: if it has a central maximum, the phase is $+1$ (i.e., the two slits are on average in phase), while if it has a central minimum the phase is -1 (the two slits are out of phase).

By measuring the correlation function for different filtering widths one obtains the relation between correlation length and spectrum width.

1.2 Simulation

For simplicity, the simulation is done with 1D instead of 2D fields. The simulation also easily allows to filter the speckle field spectrum not only with a rectangular shape but e.g. with a gaussian shape, and it therefore showcases directly the Fourier transform relation given by the Wiener-Khintchine theorem.

Otherwise, the simulation closely follows the outline traced above: in the first part, a certain number (≈ 200) of 1D speckle fields is generated and stored as `.csv` files; in a second part, the files are read and each one is fast-Fourier transformed, filtered, transformed back and used to generate an interference pattern. The patterns are then averaged over the ensemble, and the resulting averaged pattern is again stored as a `.csv`. Finally, the patterns can be read and analyzed.

2 Review of the physics

2.1 Speckle fields

A "speckle field" is an optical field with a random pattern of constructive and destructive interference, which gives rise to little dots of light called "speckles". A speckle field can be studied statistically, usually only considering first- and second-order statistics, i.e. the intensity distribution and the two-point correlation function.

The intensity correlation function of a speckle field with intensity $I(\mathbf{x}) \propto \langle |\mathbf{E}(\mathbf{x})|^2 \rangle$, and its normalized version, are given by (assuming that they only depend on the separation between the two points)

$$G(\mathbf{s}) = \langle I(\mathbf{x})I(\mathbf{x} + \mathbf{s}) \rangle, \quad g(\mathbf{s}) = \frac{G(\mathbf{s})}{\langle I(\mathbf{x})^2 \rangle}. \quad (2.1)$$

$\langle \rangle$ denotes the average over an ensemble of fields. The intensity correlation function can be computed directly from the measured intensity. It characterizes the average size of the speckles, but it doesn't contain any information about the phase relation between two points.

By contrast, the field correlation function

$$\Gamma(\mathbf{s}) = \langle \mathbf{E}(\mathbf{x})^* \cdot \mathbf{E}(\mathbf{x} + \mathbf{s}) \rangle, \quad \gamma(\mathbf{s}) = \frac{\Gamma(\mathbf{s})}{\langle |\mathbf{E}(\mathbf{x})|^2 \rangle} \quad (2.2)$$

also contains the information about the phase relation, but it cannot be computed from a direct measurement of the field. Instead, it is necessary to use interferometry to obtain the necessary information about the average phase relation.

2.2 Measuring the field correlation function

Consider a double slit, neglecting for now the finite slit width effects, probing a 1D speckle field. The far-field interference pattern is, for a slit separation s ,

$$E_{\text{screen}}(y) = E \left(-\frac{s}{2} \right) \frac{e^{ikr_1}}{r_1} + E \left(\frac{s}{2} \right) \frac{e^{ikr_2}}{r_2}, \quad (2.3)$$

where $k = 2\pi/\lambda$ and $r_{1,2}$ is the distance of the point on the screen y from the two slits. The approximation to first order $r_2 - r_1 = \sqrt{z^2 + (y - s/2)^2} - \sqrt{z^2 + (y + s/2)^2} \approx ys/2z$ is valid in the far-field regime (z is the longitudinal propagation distance). Going over to the averaged intensity,

$$\langle |E_{\text{screen}}(y)|^2 \rangle = \frac{2}{z^2} \Gamma(0) + \frac{2}{z^2} \text{Re} \left\{ \left\langle E \left(-\frac{s}{2} \right)^* E \left(\frac{s}{2} \right) \right\rangle \exp \left(\frac{2\pi}{\lambda z} y s \right) \right\}, \quad (2.4)$$

where $\Gamma(0) = \langle |E(-s/2)|^2 \rangle = \langle |E(s/2)|^2 \rangle$ is the average intensity of the speckle field. In the expression there appears the field correlation function $\Gamma(s) = |\Gamma(s)|e^{i\phi(s)}$. Therefore, modulo a z^2 factor,

$$\langle |E_{\text{screen}}(y)|^2 \rangle = 2\Gamma(0) + 2|\Gamma(s)| \cos \left[\frac{2\pi}{\lambda z} sy + \phi(s) \right]. \quad (2.5)$$

The maximum intensity I_{max} is attained when the cosine is $+1$, the minimum I_{min} when the cosine is -1 . Therefore

$$\mathcal{V} = \frac{I_{\text{max}} - I_{\text{min}}}{I_{\text{max}} + I_{\text{min}}} = \frac{|\Gamma(s)|}{\Gamma(0)} = |\gamma(s)|. \quad (2.6)$$

Finally, from the expression (2.5) it can be seen that the phase $\phi(s)$ of the correlation function determines the value of the pattern when $y = 0$. Thus, if $\langle |E_{\text{screen}}(0)|^2 \rangle = I_{\text{min}}$, then $\phi(s) = \pi$ and $e^{i\phi(s)} = -1$; if $\langle |E_{\text{screen}}(0)|^2 \rangle = I_{\text{max}}$, then $\phi(s) = 0$ and $e^{i\phi(s)} = +1$.

2.3 Finite slit width effects

In practice, the slits have a finite width (in the simulation it is of $a = 1$ mm), and therefore the interference pattern is modulated by a certain profile. In the case of coherent light, the profile is a sinc function (the diffraction pattern from a single slit), but in the case of partially coherent light the profile is more complicated. The equation (2.3) becomes, taking into account the finite width,

$$E_{\text{screen}}(y) = \int_{-s/2-a/2}^{-s/2+a/2} dx E(x) \frac{e^{ikr(x)}}{r(x)} + \int_{s/2-a/2}^{s/2+a/2} dx E(x) \frac{e^{ikr(x)}}{r(x)}, \quad (2.7)$$

where $r(x) = \sqrt{z^2 + (y-x)^2}$. Without going over the whole calculation, the resulting pattern is, modulo a z^2 factor, and neglecting the partial coherence of the field over a single slit,

$$\langle |E_{\text{screen}}(y)|^2 \rangle = a^2 \Gamma(0) \left[(I_+(y)^2 + I_-(y)^2) + 2\gamma(s) I_+(y) I_-(y) \cos \left(\frac{2\pi sy}{\lambda z} \right) \right], \quad (2.8)$$

where the functions $I_{\pm}$ can be well approximated by

$$I_{\pm}(y) = \text{sinc} \left[\frac{\pi a}{\lambda z} \left(y \mp \frac{s}{2} \right) \right] \quad (2.9)$$

if $a^2/\lambda z \ll 1$. The explicit expressions for the upper and lower modulation are obtained by setting the cosine to ± 1 ; actually, there are also effects due to the partial coherence of the field over the span of a single slit; in the code, I accounted for those effects by inserting a corrective multiplicative constant in the amplitude (this correction is only empirically justified). In the end, the expressions used to fit the upper and lower profile (respectively, to the local maxima and minima of the simulated pattern, see below) are

$$I_{\text{lower}}^{\text{upper}} = Aa^2 I_0 \left[(I_+(By)^2 + I_-(By)^2) \pm 2\mathcal{V}I_+(By)I_-(By) \right]; \quad (2.10)$$

here I_0 is the average intensity of the field incident on the slits (i.e. $\Gamma(0)$), while the fitted parameters are A , B and $\mathcal{V}$.

2.4 Computing the field correlation function

The spatial filtering of the speckle field determines its second order statistics by the Wiener-Khintchine theorem, which states that the two-point correlation function is the fourier transform of the spectral density (which is proportional to the field in the focal plane of the spatial filter). Therefore, if b denotes the width of the filtering slit, and assuming an initially δ -correlated field (so that its spectral density is a constant), it is easy to obtain eqn. (1.1). The software will also allow the option of a gaussian filtering. In that case, the field correlation function in the double slit plane would be (1.2).

3 Simulation

The simulation, as mentioned above, is articulated in three steps, managed by different windows of the GUI. The first is the generation of the speckle fields, the second is the spatial filtering together with calculation of the interference patterns using the resulting filtered speckle fields, the third is the analysis of the patterns and the calculation of the correlation function.

3.1 Generation of the speckle fields

The interface allows to choose the desired size of the statistical sample as in figure 2.

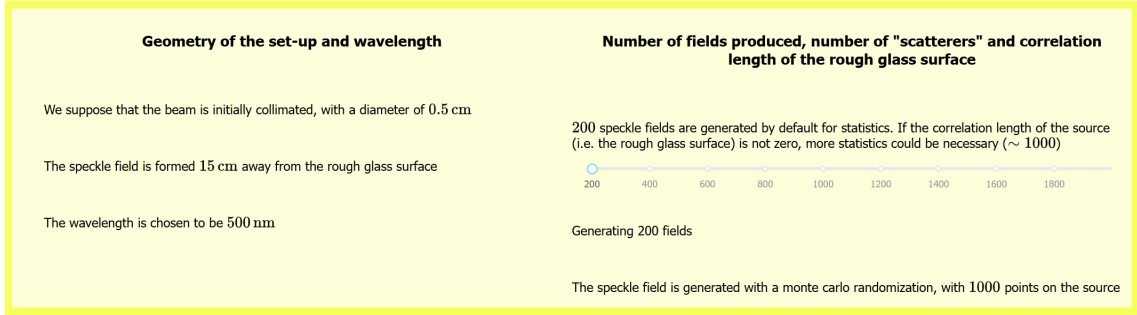


Figure 2: Choice of parameters for step 1

The fields E_{spec} are created at distance z from the source, summing spherical waves

$$E(y) = \frac{e^{ikr(x)+\phi(x)}}{r(x)}, \quad r(x) = \sqrt{(x-y)^2 + z^2}, \quad (3.1)$$

centered on points x extracted randomly in the source region, with random phases $\phi(x)$.

3.2 Spatial filtering

The spatial filtering is done by computing a DFT of E_{spec} , multiplying by the pupil function of the filter (which in the experimental reality would be a rectangle function, though the options of the simulation also allow a gaussian) and then performing the inverse DFT obtaining E_{filt} . The DFT is supposed to represent a physical spatial filter built with two lenses and a slit in their focal plane: therefore the physical meaning of the spatial frequency $\tilde{k}$ (the independent variable of the DFT) is $\tilde{k} = ky/f$, where $k = 2\pi/\lambda$ is the wavenumber, y the coordinate on the focal plane and f the focal length of the lenses. The filtering width in frequency units

is therefore $\tilde{b} = kb/f$. From (1.1) and (1.2), we see that the theoretical correlation function depends on the combination $\pi bs/\lambda f = kbs/2f = \tilde{b}s/2$ (as expected, since implementation of the filtering procedure doesn't require a definition of f nor of the physical dimension of the slit, the theoretical correlation function can be computed just from the filtering width in frequency units $\tilde{b}$).

3.3 Interference patterns

The interface allows to choose the range of the slit separation and of the filtering width as in figure 3. The pattern is computed summing spherical waves weighted

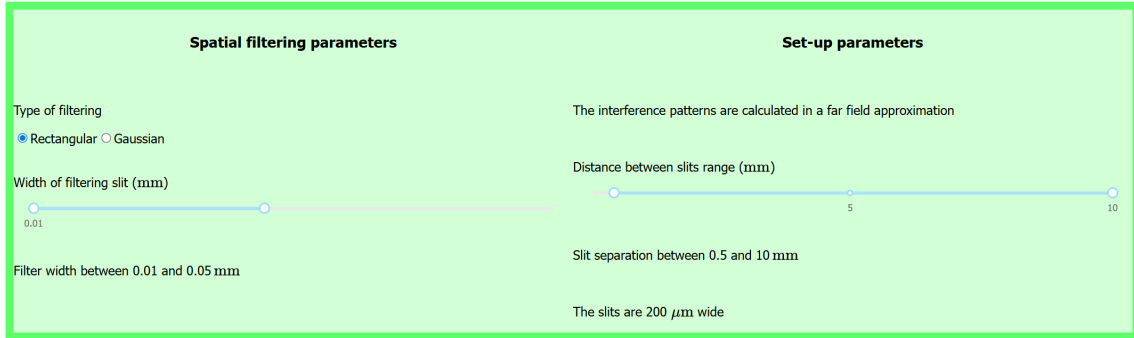


Figure 3: Choice of parameters for step 2

by the field $E_{\text{filt}}(x)$, i.e. contributes of the form

$$E_{\text{filt}}(x) \frac{e^{ikr(x)}}{r(x)}, \quad r(x) = \sqrt{(x - y)^2 + z^2} \quad (3.2)$$

for x inside the interference slits.

The interference patterns produced look like figure 4. In the figure, the modulating function of eqn. (2.8) is also shown. Since this function is calculated neglecting the effects of partial coherence over each individual slit, the actual pattern has a lower intensity.

There are three main effects which in principle have to be taken into account, as can be seen in the figure: the modulation of the cosine (due to the finite width of the slits), the loss of visibility (due to the partial coherence between the field amplitudes at the two slits) and some other effect - which could be numerical or due to partial coherence across a single slit - causing a slight and inconsistent modification of the amplitude and the width of the pattern; this last effects are unaccounted for in the model, and are described by two additional parameters in the fit: a horizontal and a vertical dilaton constant.

Example of interference pattern from partially coherent double-slit interference

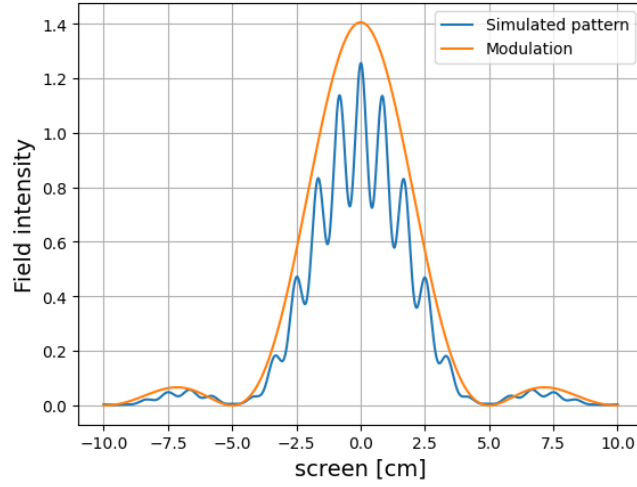


Figure 4: Typical interference pattern

3.4 Data analysis

The interface allows to visualize the interference patterns one by one, and to choose the method of analysis (mainly, how to calculate the modulation in order to remove it) as in figure 5.

The interface for data analysis allows to automatically compute the visibility of each interference pattern (previously generated and stored as csv files) as a function of both the width of the spectral density after the spatial filtering and the separation between the slits. For each fixed value of the filter width Δk , the resulting function is the absolute value of the normalized field correlation function, as per eqn. (2.6): this allows to calculate the correlation length Δl as the FWHM of the correlation function. The measured correlation function can be visualized for any value of the filter width as in fig. 5. It's then possible to plot the inverse relation between Δl and Δk .

It's possible to choose two different methods to estimate the modulation: either fit the upper profile with the modulating function in (2.8) to the local maxima, or calculate numerically the convex hull of the local maxima (fig. 6b shows the convex hull). If a greater control over the analysis of a single pattern is necessary, the interface allows also to choose manually the starting parameters for the fit of the modulation: in this case, the modulation is computed by jointly minimizing the χ^2 for both the upper and the lower profile (respectively, to the local maxima and to the local minima of the pattern; fig. 6a shows the maxima and minima as computed

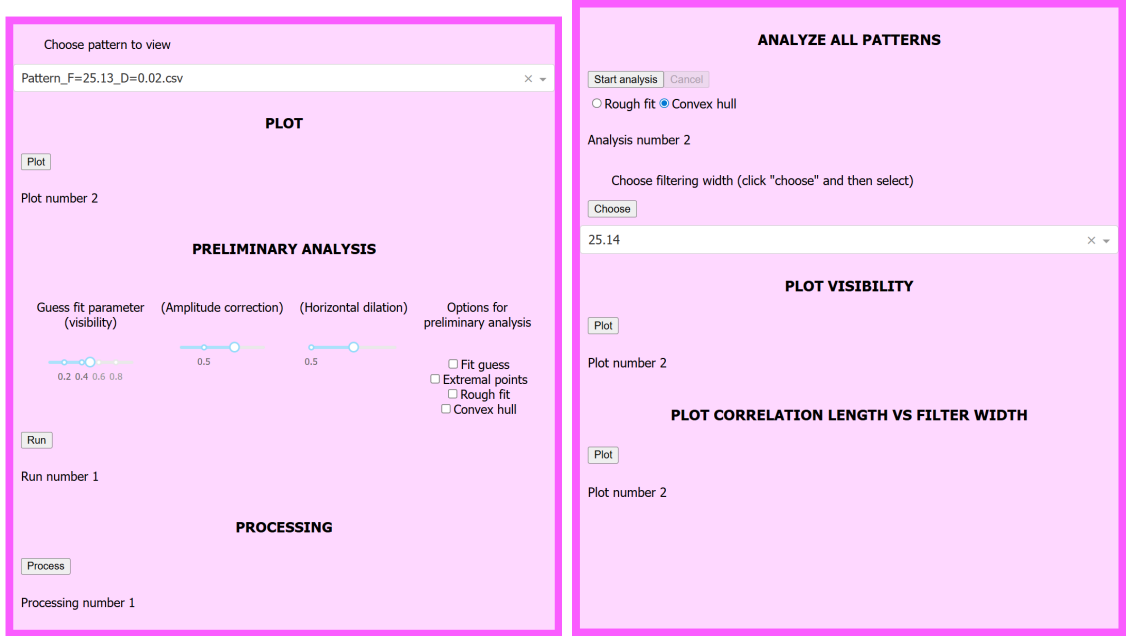


Figure 5: Data analysis interface

by the script, fig. 6c shows the fit).

Normalizing the pattern by the modulation thus calculated, a nearly (co)sinusoidal pattern results (like in figure 6d); the visibility can then be calculated using eqn. (2.6).

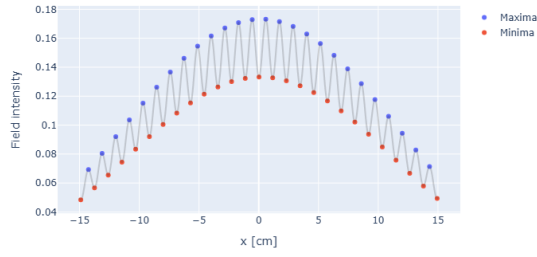
3.5 Visibility curve and phase

The visibility curve (i.e. the absolute value of the field correlation function) for a sample rectangular spatial filtering is shown in figure 8a, alongside the theoretical correlation function (1.1).

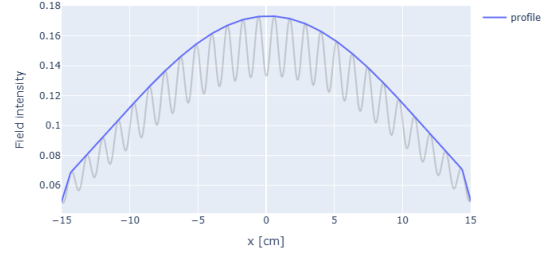
In order to determine the correlation function, it is necessary to know the absolute value (the visibility) as well as the phase. The latter is either $+1$ or -1 , depending on whether the center of the pattern is a maximum or a minimum (these two possibilities are shown in figure 7).

3.6 Relation between Δl and Δk

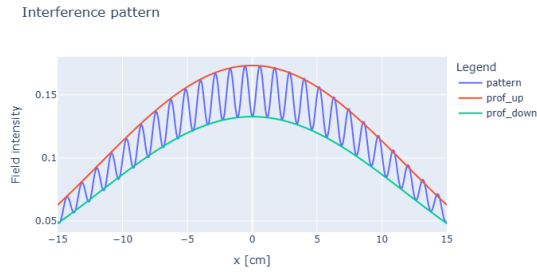
To each value of the width Δk of the filtering slit there corresponds a different width Δl of the filtered speckle field's correlation function. These two quantities are



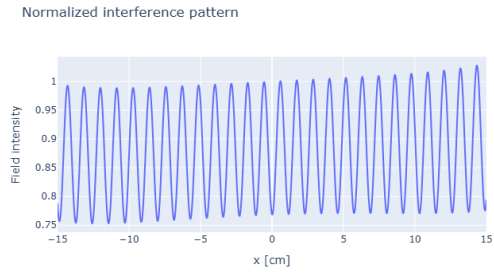
(a) Computed maxima and minima



(b) Convex hull

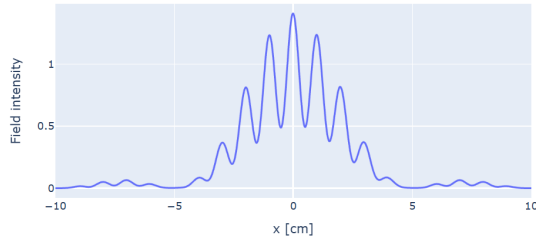


(c) Fitting of upper and lower profile

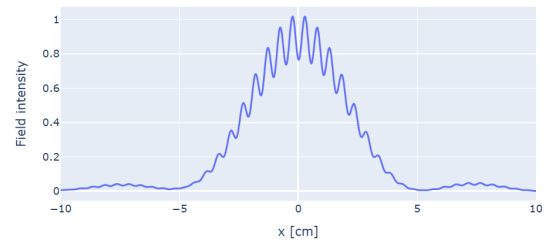


(d) Normalized pattern

Figure 6: Data analysis



(a) Phase +1



(b) Phase -1

Figure 7: The maxima and the minima swap each time the visibility becomes zero

inversely proportional, and this can be directly verified from the simulation. This is just a well-known consequence of the Fourier transform relation between the spectral density and the correlation function (the Wiener-Khinchine theorem).

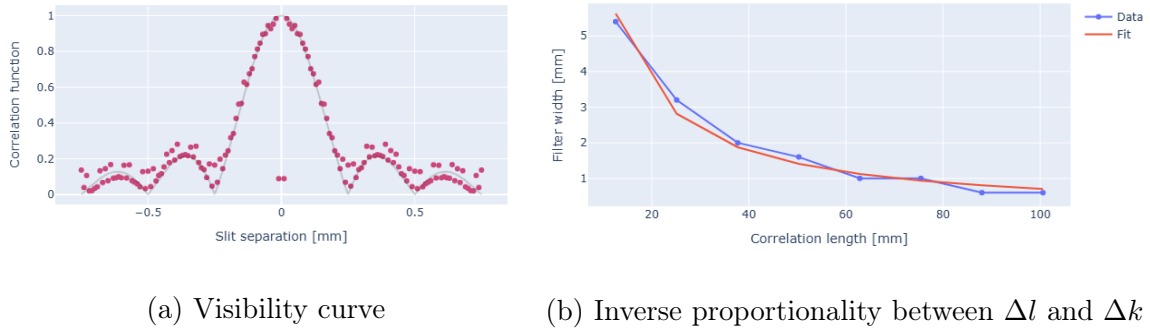


Figure 8: Results of data analysis

A Libraries

Dash, the library I used for the graphic interface, works with Plotly for the graphs, which in turn is easy to use together with Pandas for data handling.

```
import pandas as pd
from dash import Dash, html, dcc, callback, Output, Input, State, exceptions
import plotly.express as px
import plotly.graph_objects as go
```

Figure 9: Libraries: Dash, Plotly, Pandas

A.1 Brief overview of Dash

Dash allows to create a web app (run from the browser). Its structure is encoded in the "layout" function, and is made of several "components", which can either be html components or Dash core components (dcc). The layout syntax is

```
def serve_layout():
    return html.Div([
        ...
    ])
```

and all the layout components go inside the `html.Div` (the `html.Div` structure is a "box" that can be nested arbitrarily). Every component has associated properties, which for example can be a `value` (i.e. a normal python variable) or a `children` (a subcomponent). The component properties can be accessed via the component id, which is a string that identifies it. Among the dash core components are structure for input/output such as `dcc.Dropdown`, `dcc.RadioItems` etc., and also the `dcc.Graph` component, whose property is of type `figure`, which is to say a

Potly.graph_objects.Figure variable.

The user interaction happens through the `callback` functions. `callback` is a function decorator, which means it's a function that takes other functions as input and calls them inside their definition, while also performing other actions; `Dash` callbacks are applied to their argument functions with the following syntax,

```
@callback(  
    # Input and output  
    ...  
)  
def function(...): # argument function  
    ...
```

or, in the case of long callbacks (needed when the running time of the argument function is longer than 30s),

```
@app.long_callback(  
    output = [  
        ...  
    ]  
    inputs = [  
        ...  
    ]  
    ...  
    manager = ...  
)  
def function(...):  
    ...
```

The inputs and outputs of the callbacks are specified by the `id` of layout components: when the properties are changed by user input or any other way, the callback is called, and it passes the component properties to its argument function.

A.2 Notable comments on the implementation

- At the definition of a callback, its inputs, outputs and states are specified. Inputs are `Dash` components that trigger the callback upon their modification; outputs are `Dash` components that are returned to the layout at the end of the callback; states are `Dash` components which the callback can access (and on

which it depends), but that don't trigger the callback. Since many callbacks require multiple inputs (e.g. the callback for creating the interference patterns needs the range of the filtering width, the range of the slits separation and the type of filtering, as shown in fig. 3), in order to avoid triggering the callback at each modification, most parameters are passed as states, and a single confirmation button is the actual input. The property of "button" components is of type `n_clicks`.

- Some of the callbacks (generation of speckle field, interference, automatic analysis) require more than 30s, therefore it was necessary to use `long_callback`.
- By default, every callback is triggered at the opening of the app; the standard way to avoid that (since for example it's possible that the function needs to read data stored in external files that may not have been generated yet) is to raise a `PreventUpdate` exception if `n_clicks = 0`.

```
if n_clicks is None:  
    raise exceptions.PreventUpdate()
```

Figure 10: Raising `PreventUpdate` exception

- In order to increase the modularity and ease of use of the app, all the data (simulated speckles fields and interference patterns) are stored in `.csv` files and accessed later (the speckles fields are accessed by the function for creating interference patterns, the patterns are accessed by the functions for data analysis): this way the different steps can be executed independently. The same principle is applied to other parameters (setup parameters, average intensity of the fields, numerical size of the statistical ensemble) that are stored in a `.json` file.
- The `html.Div` component has a `ClassName` argument, that if defined can be used in a `.css` file to define its appearance (size, color, margin...).